

Project 1: Curves of Equal Loudness

The Python code written for this project covers two major phases. The first half allows a user to test the loudness perception of their own hearing and save this data in ELC curves. The second half uses this ELC data to perform equalization tasks on sound files. Each of these tasks builds through sequential steps using principles learned in Audio Signal Processing.

Measure Personal Threshold and ELC

Overall, the measurement phase gathers data for a total of five curves. From the softest to loudest, they are: minimal hearing threshold, soft listening, comfortable listening, medium-loud, and very loud.

1. Setup Equipment

Before beginning the data collection, the user is encouraged to set up their environment for best results, including setting the volume level of their computer to allow a full range of loudness to be heard.

2. Measure Curve: Comfortable

Each measurement phase begins with setting the volume level of the 1kHz pure tone, then that tone is used as the reference point for perceived loudness of all other pure tones to be measured. Each curve will record 24 total data points, corresponding to the center of each Bark critical band. As data is collected, it is stored in numpy arrays, which are combined into a pandas dataframe after all listening responses have been collected.

For the first measured curve, the user is asked to follow text prompts to adjust the volume level until the 1 kHz tone is at a comfortable mid-loudness listening level. The rest of the targeted frequencies are then compared to the 1kHz tone by playing alternating pairs of the target and reference tones, adjusting the amplitude of the target tone, and asking the user to select which one sounds louder. When the user's selections converge on a decibel value, the volume is lowered and this value is verified by additional user input before saving the value and iterating to the next frequency tone. In general, the code opts to start from a lower volume level than expected and increase gradually, in order to protect the user's hearing. In addition, the instructions provide options to interrupt audio playback, in case a sound is too loud. During the development of the nested functions used, it was necessary to adjust the parameters so the increments were large enough to allow the user to make steady progress, but small enough to converge at the point of perceived loudness. Trials attempted to use smaller increments at higher amplitudes, but this impeded convergence, so decibel values were used instead, and were effective.

3. Measure Curve: Minimum Threshold

To measure the minimum perceived value for each frequency, multiple series of pure tone beeps are used. Sets of ten beeps are played at incrementally decreasing decibels, and the user is asked how many they heard. The volume adjusts until the

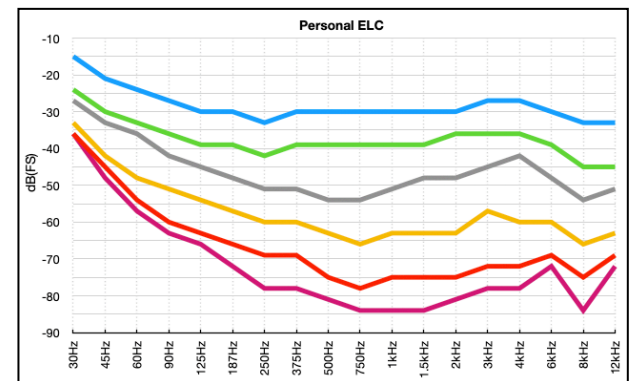
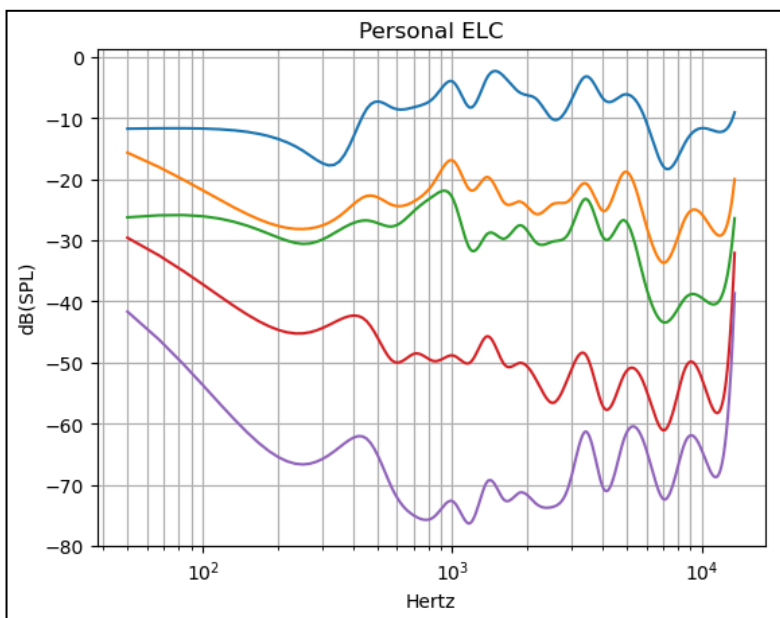
user has reported a value from 1 to 9, three separate times. This is repeated for all the critical bands.

4.-6. Measure Curves: Loudest, Soft, Medium-Loud

For each additional curve, the same functions are used as in step 2. First, the user sets the appropriate loudness for a 1 kHz tone, then pairs of tones are alternated to determine the values for the rest of the target frequencies.

ELC Curves

After completing all the measurements, the curves can be plotted, using spline interpolation to approximate values across all possible frequencies and smooth the curves. My own measurements look wavy on all curves. I also plotted results using the website shared in class (<https://www.phys.unsw.edu.au/jw/hearing.html>), which were closer to the classic contours.

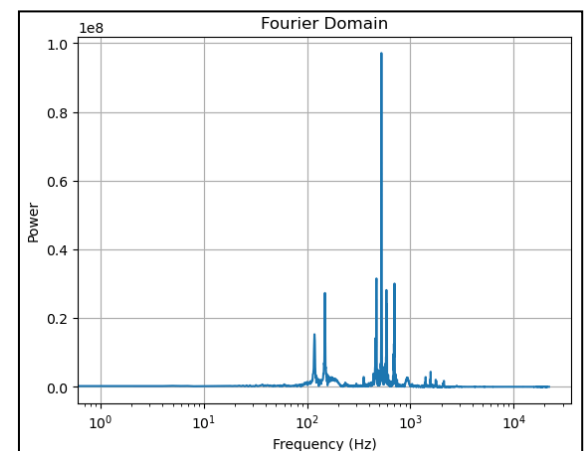


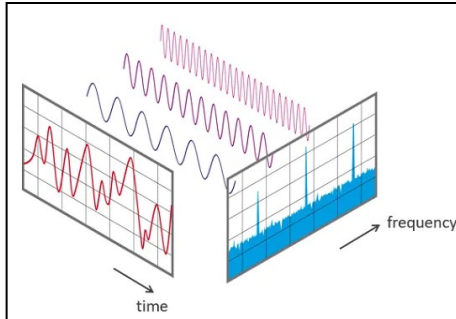
Interestingly, regardless of the test used, I consistently have heightened sensitivity around 8 kHz.

Using my computer, 16 kHz was not perceptible to me at any volume, other than a click in the headphones, possibly due to sound card limitations.

Audio Equalization

In the Audio EQ phase of the project, the saved ELC curve values are applied to transformed audio data. First, the ELC values are normalized across the full spectrum of the interpolated curves. Several functions were written to process audio files by: chunking, transforming individual chunks into Fourier domain (using fast-Fourier transform from the numpy library), multiplying factors for equivalent frequencies, then reverse-transforming back to time domain and recombining each successive processed chunk with the previous, until the full sound file is processed.





While writing these functions, I verified the data of some individual chunks by plotting in Fourier domain (shown above). I also find this domain diagram from <https://medium.com/@joshirerak123/enhancing-sound-quality-denoising-audio-with-fft-using-python-7a1d1c8c18e6> helpful in visualizing the relationship between domains.

To reduce some artifacts in the resulting audio, the chunks are created with an overlap of time, and the overlapping portions are symmetrically faded in and out as they are overlaid, for seamless integration. It was challenging to balance hyperparameters for the normalization factors, because at the extreme low and high frequencies, amplitude would need to be raised so high to achieve equality, that the middle frequencies would not be balanced. I experimented with eliminating the highest and lowest band values from the curves, and also padding with zeroes just above and below the limits of the measurements. These decisions did reduce some of the artifacts in the final processed audio.

The functions work to apply audio equalization to multiple sound files, and allow the window size and choice of ELC curve used to be varied at run time. I experimented with several sound files and multiple combinations of parameters. Some curves worked better for one kind of sound but not others. Some of the sound files were more prone to unpleasant artifacts than others. Included in the code and report are results from four sound files that were processed: a sparse electronic “instrumental” music selection, a popular music song from a band, a recording of one person speaking, and white noise. I did not find hyperparameters that made the processed band music tolerable to hear; it is full of static. However, for the other sound files, the choices made a big difference to reduce the artifacts and improve sound quality. In general, I perceive the resulting sounds as muffled. I have included side-by-side mel spectrograms of the original and processed audio, to highlight the differences from the changes applied in Fourier domain, using the personal ELC values that I gathered.

Audio Comparison: Mel-Spectrograms

